The evaluation curve

Replay Altar technical report · 2026-10-27 · results from commit `3e721c3` · GPL-3.0

Summary. For any 1v1 Warcraft III game, Replay Altar draws an evaluation curve, like the graph chess sites draw under a game: at every moment, *how often professional players in the same position went on to win* (50% is even; in the app, blue while the player is ahead and red while the opponent is), and it explains the curve's three biggest swings. The position is described by differences between the two players that can be reconstructed from the replay: army, workers, hero experience, heroes alive, expansions, resources lost and upgrades. A logistic regression whose weights change smoothly with game time was fitted on 513 curated pro games of the current patch and scored, without any refitting, on 3138 tournament games from 2022 to 2026 that it never saw. Its log loss is 0.6367 on those games (a coin flip scores 0.6931), it picks the eventual winner of 63% of games at 10 minutes and 68% at 20, and its confidence is close to honest on unseen pro games (calibration slope 0.92). This report describes the data, the model, how it was chosen and tested, and how to reproduce every number.

1. What the curve means, and what it does not

The number at minute *t* answers one question: *among pro games, how often did a player with these leads and deficits at minute t go on to win?* It is a measure of **position**, learned from players who rarely throw a lead away. It is **not** a forecast of the game being analyzed: between players who miss more chances, leads are lost more often, so the curve is too sure for them, and that is intended. A player who sees the curve fall from 60% to 20% in one minute learns that a pro in that spot would have lost ground there; that is the use the curve is built for. The two players' chances always add up to 100%.

2. Data

2.1 What a replay holds

A Warcraft III replay records only the players' commands (the game is deterministic and replays it), never the state of the game: no unit positions, no hit points, no deaths. Replay Altar reconstructs the state with WC3V, an open-source simulation of replays, with 25 changes of its own (`patches/wc3v/`, proposed upstream). The main one: WC3V reads the selection lists the replay does record (when a player changes selection, the replay lists every unit that was selected, already without the dead ones), so it knows which units exist and when they die, instead of inferring deaths from units that stop receiving orders; a train order with several buildings selected trains in each of them. Train orders the game refused never count as units, and Militia count as workers, not army. Against Warcraft III's own unit counts on 102 recorded player-games (51 games, 15 of them never used to develop the simulation), the reconstructed army value is typically off by about 23% from minute 3 on (root mean square; 32% with the simulation of the previous version); the estimated difference between the two players' armies correlates 0.90 with the real one (0.69) (`docs/DATA.md`, “How accurate the estimates are”).

2.2 Calibration set (fitting and model choice)

513 1v1 games by curated pro players on the current patch (3.0): 467 tournament games from warcraft3.info (players with a curated profile, at least 3 minutes, at most 20 games per player) and 46 W3Champions ladder games by players rated 2400 or more, collected earlier in the same way. They were collected per race (about 200 games each); a game between two pros of different races sits in two collections and is counted once. A game is used when its simulation ran to the end and its winner is known from the replay. That gives 36,504 moments (Section 3).

2.3 Validation set (scoring only)

Tournament 1v1s from warcraft3.info, created 2022 to 2026, 600 per year. Within each year the archive's pages were visited in a fixed random order, at most 4 games per page (so a year is not a few tournaments), at most 96 games per matchup and 40 per player per year, games of at least 3 minutes between two players with a profile, and never a calibration game. Downloads were made one request every 5 seconds or slower. 3,370 games were downloaded; 197 could not be simulated (mostly old W3Champions map versions that are not on warcraft3.info) and 33 had no known winner; of the 3,140 usable, 2 were left out for a broken simulation (non-finite numbers), leaving **3,138** games and 217,990 moments. These games come from older patches and from players of several levels (from the very top to semi-pros), so they test how well the model carries over, not only how well it fits. They were never used to fit or to choose the model.

The list of games (warcraft3.info replay ids, W3Champions match ids) is in `docs/win-chance/games.csv`; the replays themselves are not redistributed.

3. The position at each moment

Each game is looked at every 30 seconds from minute 1 until it ends. At each moment, for players A and B, the model sees differences A − B. 16 candidate differences were tried:

Difference	How it is measured (at time *t*)	Scale
army value	gold + lumber of the army units alive (heroes and workers excluded), valued in the form they had then	/1000
army value ratio (log)	log((A + 200) / (B + 200)) of the same army values	—
workers	workers alive (simulation)	/10
hero experience	sum over heroes of the experience of the levels reached so far (levels are exact: each level-up is a skill pick)	/1000
bases	town halls standing (expansions built and not destroyed)	—
tier	tier 2 and tier 3 finished (0–2)	—
resources spent	gold + lumber spent so far	/1000
resources lost	gold + lumber of units lost so far (heroes excluded)	/1000
food used	food in use	/10
highest hero level	level of the highest hero	—
heroes alive	heroes that have appeared minus those dead and not yet revived	—
hero deaths	hero deaths so far	—

upgrades	research finished: order time plus the game's research time for that level (repeated clicks counted once)	—
attack and armor upgrades	the same, for weapon and armor research only	—
items bought	item purchases so far	/5
creep camps (levels)	sum of the levels of the creep camps cleared so far	/20

Only what is known at time t. Every difference uses information up to *t* and nothing after it. This matters more than it seems: an earlier version estimated hero experience between two level-ups by interpolating towards the *next* level-up (and, after the last one, towards the end of the game), as the app's charts still do. That let the model see a won fight coming, because a hero about to level up was already credited with the experience. With it, the cross-validated log loss looked like 0.588; without it, it is 0.5914. The honest number is the one reported here. Cancelled research and buildings cannot be told apart in a replay and are counted as ordered.

4. The model

For a moment at time *t* with differences $\mathbf{d} = (d_1, \ldots, d_K)$ (A − B, scaled as above), the chance that A wins is

$$P(\text{A wins} \mid \mathbf{d}, t) = \sigma\left(\Sigma_k\, w_k(t) \cdot d_k\right), \quad \sigma(z) = 1 / (1 + e^{-z})$$

with no intercept. Swapping A and B flips the sign of every d_k, so P(B wins) = 1 − P(A wins) exactly, and equal players are at 50%. The weight of each difference follows the game time: $w_k(t) = \Sigma_j\, \beta_{kj} \cdot b_j(t)$, where b_j are either a straight line ($b = 1$ and t / 10 min) or “hat” functions with knots at 1, 5, 10, 15, 22 and 30 minutes (each 1 at its knot, falling linearly to 0 at the neighbouring knots, constant beyond the last), so a weight is a piecewise-linear curve through one value per knot.

Fitting. Every moment enters twice, once from each player's side (with its outcome 1 or 0), and each moment of a game with *n* moments weighs 1 / (2n), so every game counts the same whatever its length. The coefficients β minimise the weighted log loss plus a ridge penalty $\lambda\|\beta\|^2 / 2$, by Newton's method (30 iterations). A variant adds a fixed edge per race pairing (antisymmetric, so the sum stays at 100%).

Model choice used only the calibration set: 4-fold cross-validation with folds by game (a game's moments are never split between fitting and scoring). Starting from all differences, a difference was dropped whenever leaving it out did not make the cross-validated log loss worse (backward elimination). The first model started from the nine differences of an earlier study (the first nine of the table above) and was reduced the same way. Boosted decision trees (gradient boosting on log loss, depth 2–3, 150–300 trees, antisymmetrised the same way) were compared as a more flexible alternative.

5. Results

Model	Differences kept	Log loss, pro games (CV)	Log loss, validation	Brier, validation
6 differences, straight-line time weights	army value ratio (log), workers, hero experience, bases, tier, resources lost	0.6012	0.6496	0.2274

7 differences, straight-line time weights	army value ratio (log), workers, hero experience, bases, resources lost, heroes alive, upgrades	0.5927	0.6418	0.2240
7 differences, curved time weights (ridge 0.01)	army value ratio (log), workers, hero experience, bases, resources lost, heroes alive, upgrades	0.5924	0.6393	0.2232
7 differences, curved time weights (ridge 0.1) — chosen	**army value ratio (log), workers, hero experience, bases, resources lost, heroes alive, upgrades**	**0.5914**	**0.6367**	**0.2227**
7 differences, curved time weights (ridge 0.5)	army value ratio (log), workers, hero experience, bases, resources lost, heroes alive, upgrades	0.5919	0.6334	0.2219
7 differences, curved time weights, matchup edges (ridge 0.1)	army value ratio (log), workers, hero experience, bases, resources lost, heroes alive, upgrades	0.5953	0.6405	0.2240
Boosted trees (300 trees of depth 3)	all 16, and the game time	0.5953	0.6405	0.2254
Coin flip		0.6931	0.6931	0.2500

The chosen model is the one with the lowest cross-validated log loss on the pro games. On the validation set it beats the first model by −0.0129 log loss (95% interval from resampling games: −0.0168 to −0.0090). Boosted trees do no better: +0.0038 on the validation set (−0.0003 to +0.0077) and +0.0038 on the pro games. The logistic model is also small (7 curves) and lets each swing be explained difference by difference.

5.1 Calibration

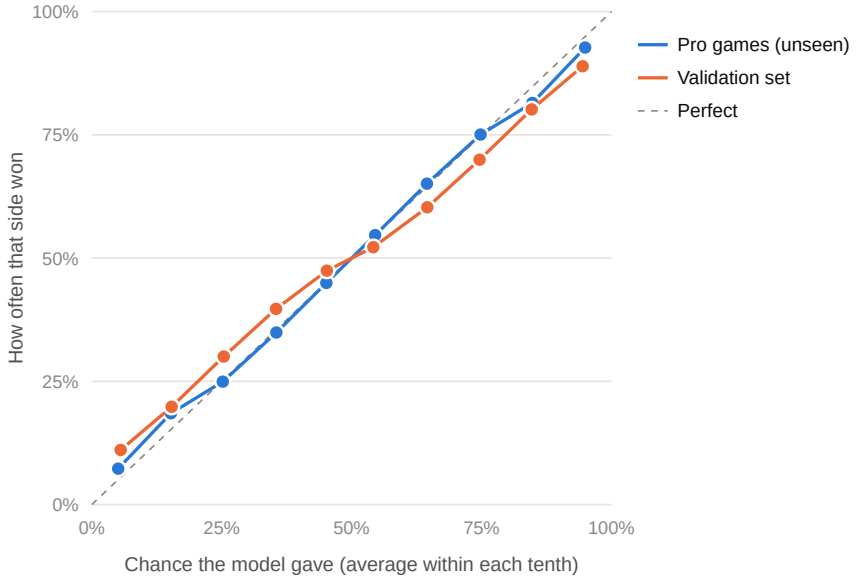


Figure 1. Moments grouped by the chance the model gave (tenths); for each group, how often that side won. Blue: pro games, each predicted by a model fitted without its fold. Orange: the validation set.

On unseen pro games the model's confidence is close to right: the calibration slope (b in P = σ(b · logit p), fitted on the predictions; 1 is perfect, below 1 too confident) is **0.92**, slightly too sure only at the extremes (when it said 95%, that side won 93%), so no correction was added. On the validation set it is

0.74: there the model is too sure, as Section 1 expects for older patches and less consistent players. A correction fitted on those games would make the curve honest for them but would also flatten the swings that show missed chances, so the curve stays a pro baseline by design.

5.2 How often the favourite wins

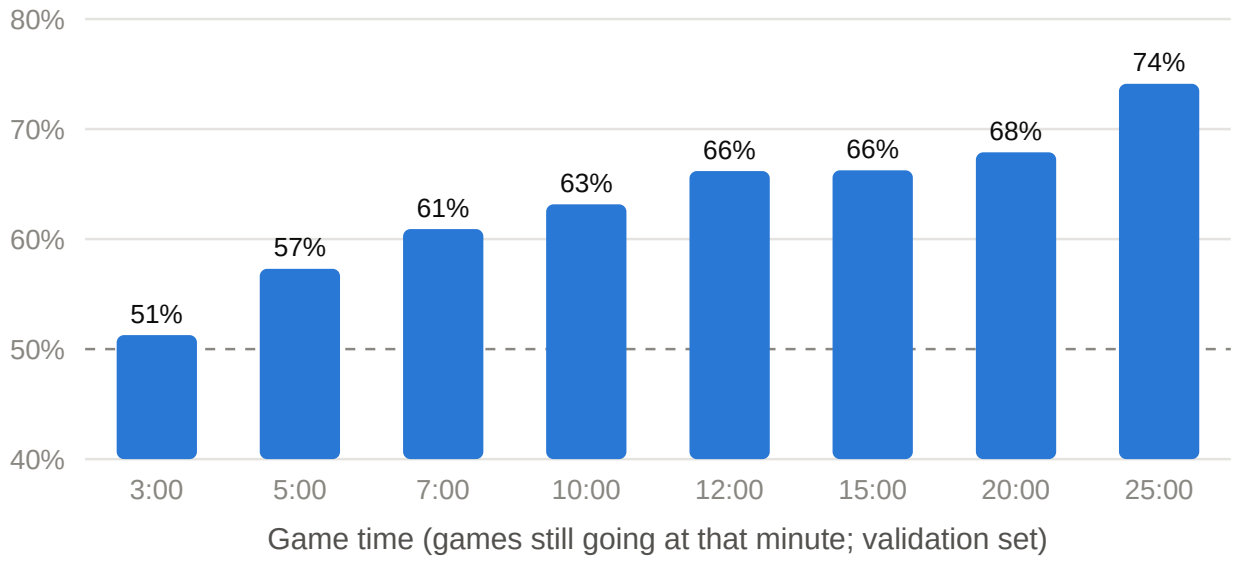


Figure 2. Share of validation games, among those still going, where the side the model favoured at that minute went on to win.

Minute	3:00	5:00	7:00	10:00	12:00	15:00	20:00	25:00
Games still going	3138	3093	2962	2663	2339	1781	1143	788
Model above 80% or below 20%	0%	4%	10%	17%	25%	35%	37%	52%

5.3 By year and matchup (validation set)

Year	Games	Log loss, chosen	Log loss, first	Favourite won at 10:00
2022	614	0.611	0.623	66% of 508
2023	601	0.639	0.652	65% of 497
2024	620	0.662	0.678	60% of 553
2025	653	0.646	0.654	62% of 554
2026	650	0.626	0.642	63% of 551

Matchup	Games	Log loss, chosen	Log loss, first	Favourite won at 10:00
HvH	301	0.610	0.627	66% of 255
HvO	321	0.613	0.628	68% of 285
HvU	317	0.640	0.662	59% of 278
HvN	331	0.648	0.680	64% of 277
OvO	297	0.646	0.648	69% of 217
OvU	316	0.666	0.670	60% of 286
OvN	328	0.618	0.630	63% of 291
UvU	287	0.655	0.659	62% of 222

NvU	316	0.644	0.651	59% of 282
NvN	324	0.629	0.642	65% of 270

The scores do not simply get worse with older games, which suggests the differences between years come more from the tournaments and players than from patch changes.

5.4 What weighs most

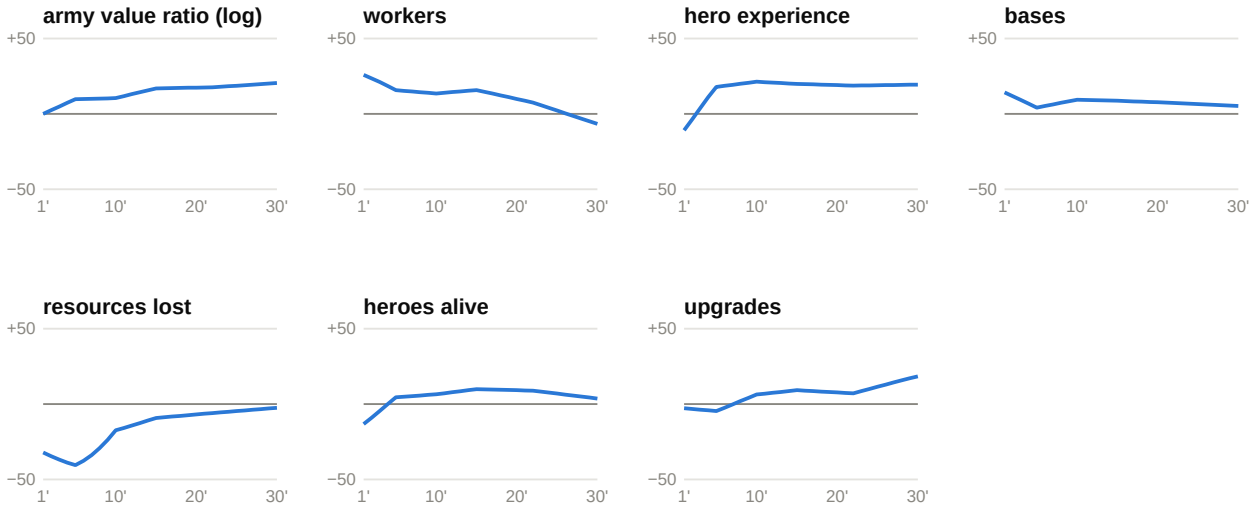


Figure 3. For each difference the chosen model uses: the change in win chance, in percentage points, for a lead of one standard deviation of that difference (across all calibration moments), at each minute, with the others even. The leads overlap (a bigger army usually comes with more experience), so the panels are read together, not one by one.

A lead of one standard deviation in…	One standard deviation	at 5:00	at 10:00	at 15:00
army value ratio (log)	0.58	+10 pts	+10 pts	+17 pts
workers	0.75	+16 pts	+14 pts	+16 pts
hero experience	0.61	+18 pts	+21 pts	+20 pts
bases	0.68	+4 pts	+9 pts	+9 pts
resources lost	1.48	−41 pts	−17 pts	−9 pts
heroes alive	0.62	+4 pts	+6 pts	+10 pts
upgrades	2.36	−5 pts	+6 pts	+9 pts

Resources lost carries a negative weight, as it should (losing more is worse). Upgrades weigh slightly against early on, when research competes with units for the same resources, and for the player from about minute 8. The standard deviation column is in the scaled units of Section 3.

5.5 How much the army value's error matters

The army value is the least certain input. To see how much its error costs, the model was refitted with extra random noise added to each player's army value at every sample, and once without the army value at all.

Army value	Log loss, pro games (CV)	Log loss, validation

As reconstructed	0.5914	0.6367
With extra noise, sd 500 per player (about the measured error again)	0.5986	0.6423
With extra noise, sd 1000 per player	0.6030	0.6451
Left out	0.6062	0.6484

Adding noise about as large as today's error again costs 0.0072 on the pro games and 0.0056 on the validation set; leaving the army value out costs 0.0148 and 0.0117. By the same slope, removing today's error altogether would gain a few thousandths at most. A better army estimate would make the app's army chart and loss numbers more accurate, but would change the curve little.

6. Explaining the swings

The app marks the three largest changes of the curve over one minute (at least 8 points, at least 90 seconds apart). For a swing from t_0 to t_1, each difference's share of the change in the model's score is $c_k = w_k(t_1)\cdot d_k(t_1) - w_k(t_0)\cdot d_k(t_0)$; the two largest in the direction of the swing (above 0.05) are named, with whom they favoured. A difference whose weight has the unexpected sign at that moment (the model is using it to correct another one) is not named, because the sentence would read backwards. Next to them the app lists what happened in the 90 seconds before t_1: fights (with the losses on each side as the report counts them), hero deaths, expansions destroyed and tiers finished. These are descriptions of the moment, not proof of cause.

7. Limitations

- **Reconstructed inputs.** Army value and losses are estimates (Section 2.1); hero deaths are approximate; cancelled orders count as done.
- **One patch of pro games.** The weights are those of patch 3.0 pro play. After a balance patch they may drift; the validation years suggest slowly.
- **No skill or matchup term.** The same leads are worth the same in every matchup (race edges did not help), and the curve does not know who is playing.
- **What the replay cannot show**: positions, vision, who is about to win a fight. The curve reacts to the results of a fight once its losses and experience show, not before.
- **Moments are not independent.** All intervals are from resampling whole games, never single moments.

8. Reproducing it

Everything is in the repository at github.com/bubunuh/replay-altar (GPL-3.0). From a Linux checkout with the simulation set up (README, “Linux”):

```
# The calibration games (or download again the ids in games.csv)
replay-altar ref refresh --all
# The validation games (one request every 5 seconds: several hours), then re-summarized
node scripts/download-validation.ts --target 3000
node scripts/observer/resummarize.ts --group validation
# Fit, choose and score: docs/WIN-CHANCE.md, data/winchance.json, results.json, games.csv
node scripts/win-chance.ts
# This report (HTML, then PDF through Electron)
node scripts/win-chance-report.ts
```

The model's inputs and the curve are computed by one file shared by the research script and the app, `packages/rules/src/winchance.ts`; the fitted model the app ships is `data/winchance.json`. The army value measurement is in `scripts/observer/` and `docs/DATA.md`. Replays are not redistributed: the games are listed by id in `games.csv` and can be downloaded again from warcraft3.info and W3Champions.